

Air Quality Forecasting System

Muhammad Abdullah

Karachi, Pakistan

Dashboard: [Click Here](#)

GitHub: github.com/abdullah

Abstract—Traditional forecasting models predicting mean daily particulate matter (PM2.5) often fail to warn users of severe, intra-day pollution spikes. This paper presents an end-to-end Air Quality Index (AQI) forecasting system designed to provide a reliable early-warning mechanism. By engineering localized time-series features, mitigating hardware sensor anomalies, and evaluating multiple architectures—including Ridge Regression, XGBoost, and a Multi-Layer Perceptron (MLP) with L2 Regularization—the system accurately predicts pollution scenarios up to 72 hours in advance. The decoupled architecture leverages Hopworks for feature storage, GitHub Actions for continuous integration, a FastAPI backend with dynamic in-memory model updates, and a Next.js frontend, ensuring continuous automated operation.

Index Terms—Air Quality Forecasting, MLOps, Deep Learning, XGBoost, Feature Store, FastAPI, Explainable AI

I. INTRODUCTION

Air pollution is a highly volatile, non-linear phenomenon driven by weather patterns, industrial activity, and human behavior. In consumer-facing applications, predicting a mathematically accurate “average” daily AQI can be profoundly misleading; averaging 20 hours of clean air with a 4-hour severe smog event yields a “moderate” warning, effectively masking the true health risk from the user.

This project details the development of a Machine Learning Operations (MLOps) pipeline designed specifically to track and forecast PM2.5 trajectories. By prioritizing rigorous evaluation across multiple algorithmic archetypes and deploying an automated model selection pipeline, the system reliably transforms raw meteorological API data into an actionable health safety product.

II. EXPLORATORY DATA ANALYSIS (EDA)

Comprehensive EDA was conducted on historical hourly data across globally diverse cities (e.g., Lahore, London, Beijing). The analysis revealed critical insights that dictated the subsequent feature engineering phase:

A. Seasonality and Human Trends

Analysis of temporal distributions highlighted distinct environmental patterns. Monthly boxplots exhibited a pronounced U-shape, confirming the severity of winter temperature inversions trapping particulate matter close to the surface. Day-of-week analysis showed marked PM2.5 dips on weekends, quantifying the exact impact of weekday industrial operations and commuter traffic.

B. Intra-Day Volatility and Peak Discrepancy

An experiment comparing daily mean PM2.5 against daily maximum PM2.5 proved that intra-day volatility is the primary forecasting challenge. Days with safe or moderate averages frequently contained massive $300 + \mu\text{g}/\text{m}^3$ spikes. This discovery demonstrated that flat averages hide extreme events, mandating the inclusion of dynamic rate-of-change metrics and rolling momentum features to help the models anticipate sudden atmospheric deterioration.

III. DATA ENGINEERING & FEATURE PIPELINE

A robust data transformation layer was engineered to continuously extract 14-day rolling windows of data via the Open-Meteo API, utilizing exponential backoff protocols to ensure reliability against network failures.

A. Forward-Fill Imputation Strategy (Synthetic Today)

A critical challenge in real-time forecasting is computing daily aggregations for the current, incomplete day. To resolve the discrepancy between available future weather forecasts and absent future pollution data, the pipeline implements a forward-fill mechanism.

The system queries an additional 24 hours of future weather forecast data (e.g., expected temperature and wind speed). Because future PM2.5 data cannot be observed, the pipeline applies a chronological forward-fill function (`.ffill()`), carrying the last recorded PM2.5 measurement into the unobserved future hours. This seamlessly stitches past observed pollution data with predicted meteorological patterns, allowing the pipeline to generate safe daily aggregates for the current day without missing values.

B. Hardware Anomaly Mitigation

Before aggregation, the system scans for isolated, physically impossible sensor errors. For instance, PM2.5 jumping by $850 \mu\text{g}/\text{m}^3$ in a single hour and immediately dropping is a hardware glitch, not an atmospheric event. Instead of dropping these rows and breaking time-series continuity, the system identifies bi-directional spikes exceeding $300 \mu\text{g}/\text{m}^3$, neutralizes them to NaN, and applies linear interpolation to mathematically smooth the anomaly.

C. Aggregation and Macro-Momentum

The cleaned hourly data is aggregated into daily means. The pipeline then computes essential time-series features: 3-day and 7-day rolling averages, alongside a derivative rate-of-change feature utilizing lag mechanics to measure pollution

trajectory. Target vectors (1-day, 2-day, and 3-day ahead horizons) are shifted and padded with a safe numerical constant (-1.0) to prevent schema crashing during cloud insertion.

IV. METHODOLOGY & MODEL ARCHITECTURE

A. Cross-Spatial Global Modeling Strategy

Initial iterations experimented with training a distinct “Local Model” for each city. While highly specific, this approach suffered from data scarcity regarding extreme smog events in cleaner cities. The final architecture utilizes a “Global Model,” dropping geographic identifiers and concatenating data from all cities. This cross-pollination allows the model to learn the fundamental physics of a smog event in Beijing and intuitively apply those lessons to emerging weather patterns in Lahore.

B. Model Zoo and Cross-Validation

The training pipeline provisions separate champion models for 1-day, 2-day, and 3-day horizons. A Model Zoo is evaluated, comprising Ridge Regression, Random Forest, XGBoost, and a Deep Learning Multi-Layer Perceptron (MLP). Each algorithm undergoes rigorous hyperparameter tuning via GridSearchCV coupled with TimeSeriesSplit to prevent temporal data leakage.

C. Implicit Feature Selection via L2 Regularization

Deep learning architectures typically overfit when exposed to uninformative features. Rather than hard-coding separate, dimensionally reduced datasets for the MLP, the system relies on L2 Regularization. By passing the α penalty parameter through the hyperparameter grid, the MLP is mathematically forced to decay the weights of uninformative features toward zero. This acts as an implicit feature-selector, retaining the predictive power of the core feature matrix without requiring secondary PCA pipelines.

D. Segmented Evaluation Metrics

The dataset exhibited severe class imbalance, with 99.4% of days classifying as “Normal” ($< 100 \mu\text{g}/\text{m}^3$). Standard overall metrics like RMSE easily mask poor performance during rare, hazardous events. A custom evaluation layer was engineered to segment predictions into severity buckets. As shown in Table I, the system maintains highly accurate Mean Absolute Error (MAE) during normal conditions while bounding the error margins during moderate events.

TABLE I
SEGMENTED MAE ACROSS FORECASTING HORIZONS

Horizon	Condition	MAE ($\mu\text{g}/\text{m}^3$)
1-Day	Normal (< 100)	4.77
	Moderate (100-200)	26.04
2-Day	Normal (< 100)	6.37
	Moderate (100-200)	38.53
3-Day	Normal (< 100)	6.82
	Moderate (100-200)	41.09

E. Explainable AI (SHAP) Integration

To ensure interpretability, the pipeline dynamically generates SHAP (SHapley Additive exPlanations) values based on the winning architecture. For Deep Learning champions, the system utilizes a 50-centroid K-Means summarization of the training set before applying the KernelExplainer, exponentially reducing the computational overhead required to interpret the neural network.

V. SERVERLESS MLOPS ARCHITECTURE

The system features a completely decoupled, zero-downtime architecture, illustrated in Figure 1.

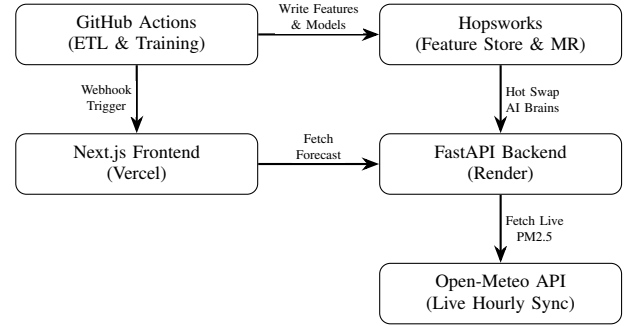


Fig. 1. Fully Decoupled Serverless Architecture

A. Data Pipeline & Feature Store

Daily ingestion scripts extract incremental weather and pollution data, execute the engineering transformations, and push the data to Hopworks. Hopworks acts as the central state-manager, housing both the materialized Feature Groups and the versioned Model Registry.

B. FastAPI Backend & Dynamic Reloading

The backend inference engine is decoupled from the training environment. It utilizes an “Integrated Refresh Cycle,” caching pre-computed ML forecasts in RAM alongside live data fetched hourly from Open-Meteo.

To orchestrate asynchronous updates, GitHub Actions fires a secure webhook to the FastAPI server upon the completion of a training run. The backend intercepts this webhook, initiates a 25-minute sleep cycle to allow the managed Hopworks Spark clusters to fully materialize the offline feature views, and then seamlessly reloads the newly versioned models into RAM without dropping incoming user requests.

C. Frontend Deployment

The user interface is deployed as a Next.js application on Vercel. Because the backend handles the mapping of raw PM2.5 to the standard Air Quality Index, the frontend remains entirely stateless, existing purely to deliver real-time, 3-day forecasts to the end user.

VI. CHALLENGES AND TECHNICAL SOLUTIONS

The development of a project using a serverless stack for the first time introduced several compounding data and architectural challenges. The following section details the primary technical obstacles encountered and the engineering strategies deployed to resolve them.

A. Real-Time Data Continuity (*The Incomplete Day*)

Challenge: Operationalizing the model for live inference required aggregating rolling features for the current day. However, querying the Open-Meteo API mid-day yields an incomplete data matrix; historical PM2.5 is available up to the current hour, but future hours are absent, causing ‘NaN’ propagation that breaks daily statistical aggregations.

Solution: A chronological forward-fill imputation strategy was developed. By fetching an additional 24 hours of predicted meteorological data (which is available in advance) and applying a forward-fill (‘.fill()’) to the last recorded PM2.5 observation, the pipeline successfully simulates a complete 24-hour block. This allows the system to compute safe daily rolling means for live inference without data leakage.

B. Time-Series Integrity vs. Hardware Anomalies

Challenge: Low-cost air quality sensors frequently suffer from hardware anomalies, outputting physically impossible PM2.5 spikes (e.g., an instant increase of $850 \mu\text{g}/\text{m}^3$ followed by an immediate drop). Standard preprocessing techniques involve dropping these anomalous rows; however, removing rows breaks time-series continuity, which corrupts the rolling average and momentum calculations.

Solution: Instead of dropping rows, an anomaly detection algorithm was implemented to flag bi-directional spikes exceeding $300 \mu\text{g}/\text{m}^3$. These specific anomalies are neutralized to ‘NaN’ and replaced via linear interpolation, preserving the chronological index required for safe time-series momentum calculations.

C. Feature Store Schema Validation Failures

Challenge: Shifting the target variable to create 1-day, 2-day, and 3-day forecasting horizons inherently generates trailing ‘NaN’ values at the end of the temporal dataset. During automated CI/CD pipeline executions, attempting to upload these shifting target columns caused the downstream PySpark engine in Hopsworks to crash due to strict schema validation failures.

Solution: A localized padding step was introduced immediately before the cloud insertion phase. Missing target vectors are populated with a safe numerical constant (-1.0) to satisfy the PySpark schema constraints. These synthetic padding values are subsequently filtered out during the data-fetching phase of the training pipeline, ensuring they never corrupt the training distributions.

D. Overfitting in Neural Networks

Challenge: While tree-based ensembles (XGBoost, Random Forest) naturally execute feature selection via Information Gain—ignoring uninformative variables—the Multi-Layer Perceptron (MLP) suffered from the Curse of Dimensionality. When exposed to the complete feature matrix, the neural network attempted to map noise, leading to significant overfitting on the training data.

Solution: Rather than engineering and maintaining a secondary, dimensionally reduced dataset (e.g., via PCA) exclusively for the neural network, the system leveraged L2 Regularization. By injecting an ‘alpha’ penalty parameter into the GridSearchCV hyperparameter tuning space, the MLP mathematically decays the weights of uninformative features toward zero, achieving implicit feature selection natively.

E. Zero-Downtime Asynchronous Model Inference

Challenge: Updating the production models required downloading new ‘.pkl’ artifacts from the Hopsworks Model Registry. Executing this synchronously during an incoming client request resulted in extreme latency and API timeout errors on the Next.js frontend.

Solution: The FastAPI backend was decoupled from synchronous inference updates. It utilizes a continuous RAM cache to serve instantaneous predictions. When GitHub Actions completes a retraining cycle, it fires a secure webhook to the backend. The backend triggers a background asynchronous task, waits 25 minutes for cloud feature views to materialize, and then silently hot-swaps the new models into active memory, guaranteeing zero downtime for end users.

VII. CONCLUSION

This project successfully bridges the gap between academic data science and consumer product engineering. By mitigating hardware anomalies, engineering localized time-series metrics, and deploying a self-optimizing Model Zoo, the system acts as a highly reliable forecasting mechanism for atmospheric events. The fully automated, webhook-driven MLOps pipeline ensures the system extracts data, trains multiple architectures, evaluates edge-case smog events, and dynamically updates production models daily with zero manual intervention.